

ALEXIS E. LUZON

Full Stack Developer

alexiseluzon.vercel.app • [linkedin.com/in/alexis-luzon-63849238b](https://www.linkedin.com/in/alexis-luzon-63849238b) • alexiseluzon@gmail.com • Philippines

PROFESSIONAL SUMMARY

Full Stack Developer transitioning from a 7-year career in food service operations, with the last 4 years as a Crew Trainer, into hands-on software development. Has independently built and deployed 10+ full-stack projects — including a live capstone PWA (PediaLink) and a Next.js/n8n/PostgreSQL internal-tool demo — covering React/Vue/Next.js frontends, typed Node/Express and Next.js backends, relational database design, automation-platform integration, and CI/CD-gated testing pipelines.

TECHNICAL SKILLS

Languages: JavaScript/TypeScript, Python, PHP, SQL

Frontend: Angular, Vue 3, Ionic, React, Next.js, Flutter, React Native, Expo, React Native Web, Tailwind CSS, HTML, Shopify (Liquid, theme customization), GraphQL, Apollo Client, SvelteKit

Backend: Laravel, REST APIs, Node.js, Express, GraphQL, Apollo Server, Hono, NestJS, Firebase Admin SDK, FastApi, Convex, Rate Limiting

Technical SEO: Sitemap/robots.txt architecture, canonical tags, JSON-LD structured data, Core Web Vitals, programmatic/dynamic page generation, Google Search Console, Rich Results Test

Database: MySQL, MongoDB, PostgreSQL, pgvector, Prisma, Neon (serverless PostgreSQL), Drizzle ORM, SQLAlchemy (async), Alembic (migrations), Convex (real-time database)

Tools: Git, GitHub, Husky (pre-commit hooks), Azure DevOps (CI/CD pipelines), VS Code, Hostinger, Vercel, Pusher, Laravel Echo, JWT, Render, Github Actions (CI,CD), ESLint, Prettier, Cursor (AI-assisted development), Expo Router, Expo push notifications (Expo Notifications API)

Mapping/Visualization: CSS, ArcGIS, Maps, SDK for JS, Chart.js

Testing: Jest, Supertest (integration), Playwright (E2E), Vitest, Testing Library (React), React Native Testing Library, mocked-LLM testing, convex-test (integration)

Auth & Payments: Firebase Auth (Admin SDK token verification), Google OAuth 2.0 (ID token verification), Stripe (Elements, PaymentIntents, webhooks), JWT (passlib/bcrypt password hashing), Clerk

CRM/Integrations: HubSpot API

CI/CD: GitHub Actions

AI & Automation: LLM APIs (OpenAI-compatible, Groq), structured outputs (JSON schema + zod validation), function/tool calling, agentic workflows, RAG, embeddings (Transformers.js), pgvector, prompt engineering, prompt-injection mitigation, n8n workflows, webhooks (Slack-compatible)

PROJECTS

DelivEasy — Full-Stack Delivery & Inventory Platform — deliver-easy.vercel.app or github.com/alexiseluzon/delivereasy

A full-stack delivery platform with a vendor web dashboard and customer/rider mobile app, with role-based access across four roles and a server-enforced order workflow.

- Built a FastAPI backend (SQLAlchemy 2.0 async, Supabase/Postgres) with JWT authentication, role-based access control (admin/vendor/rider/customer), and ownership-scoped authorization enforced at the endpoint level
- Designed an order lifecycle state machine (pending → confirmed → preparing → out for delivery → delivered) with server-enforced valid transitions and race-safe rider order-claiming via conditional SQL updates
- Built a cross-platform mobile app (Expo Router, React Native) covering customer shopping/checkout and a separate rider delivery-acceptance flow, with SecureStore-backed session persistence
- Integrated Expo push notifications (order-status updates for customers, new-delivery broadcasts for riders) via a backend notification service, with safe fallback handling for Expo Go's SDK 53+ push limitations
- Wrote Pytest integration tests (auth, orders, role permissions, concurrent-claim race conditions) against an isolated Supabase test database, and Jest unit tests across both Next.js and Expo apps
- Deployed backend to Render (Python 3.12, Alembic auto-migration on deploy) and web dashboard to Vercel, with environment-based CORS configuration

LearnRec — Cross-Platform Learning Recommender

— learnrec.vercel.app or

github.com/alexiseluzon/LearnRec

A learning-resource recommender for iOS, Android, and web from one Expo codebase, with tag-based recommendations from a user's highest-rated resources.

- Built a NestJS API (Prisma, PostgreSQL on Neon) with JWT authentication, Google ID-token verification, and DTO validation that rejects unknown fields
- Designed rule-based recommendations from tags shared with highly rated resources, with one rating per user per resource enforced by a unique database constraint
- Shipped one Expo (React Native) codebase to iOS, Android, and web via React Native Web, with a SecureStore/localStorage token fallback and a cross-platform alert helper
- Wrote Jest, Supertest, and React Native Testing Library tests, with GitHub Actions CI running lint, migrations, unit, and E2E tests against a Postgres service
- Deployed the API to Render (migrations at build time, health check, environment-based CORS) and the web build to Vercel, with a demo account for reviewers

Parts Marketplace — partsmarketplace.vercel.app or github.com/alexiseluzon/Parts-Marketplace

A full-stack parts inventory marketplace with Supabase Auth, Row Level Security, and role-based access control, migrated from a custom JWT/Neon stack to Supabase.

- Migrated authentication from custom JWT/httpOnly cookies to Supabase Auth, verifying Supabase-issued JWTs against its JWKS endpoint on every GraphQL request
- Designed a role-based access control system (user/admin) enforced at two layers: resolver-level checks in the GraphQL API and Postgres Row Level Security policies as a database-level safety net
- Built a Postgres trigger auto-provisioning a profile row (with default role) on every Supabase Auth sign-up, keeping app-level user data in sync with auth.users
- Built a GraphQL API (Apollo Server, Express) with full CRUD resolvers, ownership-scoped queries, and an admin bypass mirrored between application logic and RLS policies
- Built the frontend with React, TypeScript, and Apollo Client, attaching live Supabase session tokens per-request via an Apollo auth link, with role-aware UI (admin-only data visibility)

TaskFlow — frontendtaskflow.vercel.app or github.com/alexiseluzon/taskflow-frontend

- Built TaskFlow, a full-stack MERN task management application, handling the complete development lifecycle from architecture to deployment
- Implemented JWT-based authentication and role-based authorization, protecting REST API endpoints with middleware-driven access control
- Designed MongoDB schemas with compound indexing for efficient querying, plus pagination and centralized error handling for production-grade reliability
- Built a responsive React frontend with protected routing, custom hooks for data fetching, and reusable components
- Wrote unit and component tests (Vitest, Testing Library) covering core business logic and UI behavior
- Deployed as serverless functions on Vercel with MongoDB Atlas, configuring environment-specific database connection pooling

Willow Demo — Full-Stack Follow-up Tracker

— willow-demo.vercel.app/tickets or

github.com/alexiseluzon/Willow-Demo

Full-stack Next.js App Router app with an AI layer: LLM triage, RAG similar-ticket search, and a tool-calling agent

- Built LLM auto-triage: new tickets from the UI or an n8n webhook are categorized, prioritized and summarized via JSON-schema structured output validated with zod; failures fall back to manual triage with one-click retry
- Implemented RAG similar-ticket search using local embeddings (MiniLM, 384-dim) stored in pgvector on Neon, queried by cosine similarity
- Built a tool-calling agent (search similar tickets, add notes, propose status changes) scoped to the current ticket, with user confirmation before any write and prompt-injection safeguards
- Added Slack-compatible alerts for high/urgent tickets, plus timeouts, retries with backoff, rate limiting and input validation
- Designed a relational schema (Ticket → Note, cascade delete) with Prisma and PostgreSQL (Neon), using pooled vs. direct connections for runtime vs. migrations
- Built dual write paths (Server Actions for the UI, REST routes for n8n) sharing one validation module
- Tested with Jest (mocked LLM), Supertest, Playwright E2E and GitHub Actions CI

Hapines — hapines.vercel.app or github.com/alexiseluzon/hapines

- Built a full-stack AI chat app with session-scoped conversation history and a prompt-engineered system persona enforcing safety guardrails (crisis redirection, no medical/diagnostic claims)

- Designed a Postgres schema (Drizzle ORM) with indexed session-based message storage and an automated 30-day retention policy via scheduled cron cleanup
- Implemented CI/CD with GitHub Actions (typecheck, lint, unit, E2E) gated on branch protection; enforced code quality via ESLint/Prettier across both apps
- Wrote 29 tests (Vitest, Playwright) covering chat flow, session history propagation, rate limiting, and prompt safety behavior
- Deployed both services independently on Vercel, including a serverless Hono adapter and scheduled cron job for automated data cleanup

Onboarding Quiz Flow — Firebase Auth + Stripe — onboarding-demo-front.vercel.app or github.com/alexiseluzon/onboarding-demo

- Built a 9-step onboarding quiz in React (Vite, React Router) with resumable progress, saving each step to a Node/Express API backed by Prisma and PostgreSQL (Neon)
- Integrated Firebase Auth (email/password + Google SSO) with server-side ID token verification via Firebase Admin SDK
- Implemented Stripe Elements for payment, server-defined pricing, and a signature-verified webhook confirming payment status independent of client redirects
- Added a non-blocking HubSpot contact push on quiz completion, Jest/Supertest backend tests, Playwright E2E tests, and CI via GitHub Actions

Knō Demo — Real-Time Learning Marketplace — kno-demoo.vercel.app or github.com/alexiseluzon/kno-demo

A real-time learning session board with authenticated posting, built on the Next.js, Convex, and Clerk stack.

- Built a Next.js (App Router), TypeScript, and Tailwind CSS frontend with Convex real-time queries and Clerk authentication
- Enforced validation, ownership-scoped deletes, and per-user rate limiting in Convex mutations, and kept host identifiers out of client responses
- Added accessibility (WCAG 2.1 AA basics), SEO (metadata, JSON-LD, sitemap, robots), confirm dialogs, toasts, and inline form validation
- Wrote Jest unit tests, Vitest + convex-test integration tests (auth, validation, rate limit, ownership), and Playwright E2E tests
- Set up GitHub Actions CI (lint, typecheck, tests, build) and Husky pre-commit hooks; deployed to Vercel with a production Convex deployment

CleanCo — Programmatic SEO Demo — cleanco-programmaticseo.vercel.app or github.com/alexiseluzon/Programmatic-SEO-Demo

A Next.js programmatic SEO system generating 60+ indexable location pages from a single data source, built to solve real indexing and crawlability problems at scale.

- Built 60+ static-generated location pages (state → city hierarchy) using Next.js generateStaticParams, scalable to thousands with no code changes
- Implemented auto-generated sitemap.xml, robots.txt, and unique canonical tags per page to resolve real indexing/crawlability issues
- Added per-page LocalBusiness JSON-LD structured data, validated against Google's Rich Results Test
- Verified live indexability via Google Search Console (confirmed crawlable, indexing in progress) and Core Web Vitals via PageSpeed Insights

Regional Wells Data Dashboard — welldata.vercel.app or github.com/alexiseluzon/Regional-Wells-Data-Dashboard

A public-facing data exploration dashboard consuming a live ArcGIS REST FeatureServer, built to demonstrate geospatial visualization and accessibility-compliant frontend development.

- Integrated ArcGIS Maps SDK for JavaScript with dynamic query-builder filters (region, category) updating the map via live REST queries
- Built a multi-series Chart.js visualization aggregating live-queried data, plus CSV/JSON export
- Ensured WCAG 2.1 AA compliance (skip links, focus states, aria-live regions), verified with automated axe-core accessibility scanning
- Extracted core logic into pure functions covered by 17 Jest unit tests

Feedback Tracker — feedback-track.vercel.app or github.com/alexiseluzon/Feedback-Tracker

A client feedback and feature-request tracker with a full status workflow, built to match a Vue/Node/Express/Prisma/PostgreSQL job stack.

- Built end-to-end CRUD workflow (create, status update, delete) with Vue 3, TypeScript, and a thin-controller Express API backed by Prisma and PostgreSQL

- Implemented optimistic UI updates with automatic rollback on request failure, plus centralized server-side input validation and error handling
- Added disabled-state form validation and confirmation dialogs before destructive actions
- Deployed client and server as separate Vercel projects with environment-based configuration for CORS and API URLs

PageSpeed — pagespeed-demo.vercel.app or github.com/alexiseluzon/PageSpeed-Demo

- Diagnosed and resolved a deliberately unoptimized site to demonstrate Core Web Vitals improvement workflow
- Reduced image payload 99% (48MB→~500KB) via WebP conversion, resizing, and responsive srcset delivery
- Deferred render-blocking JS/CSS, removed unused dependencies (jQuery) and dead CSS rules
- Fixed invalid JSON-LD Organization schema to pass Google Rich Results Test with zero errors
- Mobile Performance 64→90, Desktop Performance 65→99, SEO 82→100 (PageSpeed Insights)

Testing & CI/CD Pipeline Demo — github.com/alexiseluzon/Testing-and-CI

A full testing pipeline demonstrating unit, integration, and E2E testing with automated git-hook and CI enforcement.

- Built unit tests (Jest) and integration tests (Supertest) against a live Express app, plus E2E tests (Playwright) driving a real browser
- Configured Husky pre-commit hooks to block commits when tests fail, verified live by intentionally breaking a function and confirming the commit was rejected
- Wrote an Azure DevOps pipeline (azure-pipelines.yml) to run the full test suite on every push, gating deployment on passing tests

PediaLink — pedia-link.vercel.app or github.com/alexiseluzon/PediaEaseFrontend

A Progressive Web App for pediatric health management — capstone project, deployed live with SSR and offline support.

- Designed a 138-rule clinical vaccine-scheduling engine encoding the Philippine pediatric immunization schedule, with dependency chains, interval windows, and automatic series-reset logic for missed boosters
- Built real-time one-to-one messaging via a hand-rolled Pusher HTTP API integration (manual HMAC signing, cURL-to-stream-context fallback) with read/unread tracking
- Implemented vaccination record management with field-selectable smart search, PDF export (jsPDF), and printable formatted records
- Migrated critical endpoints from Laravel to a custom PDO REST API layer with hand-rolled JWT authentication, including an admin analytics dashboard with configurable reporting periods and rate calculations
- Built with Angular 19 (PWA, SSR, service worker), Laravel 11, custom PHP/PDO API, MySQL — deployed on Render and Vercel

AssetTrack — github.com/alexiseluzon/Inventory-Laravel-Inertia

- Implemented borrowing, returns, damage reporting, and repair tracking with automatic overdue-status detection via request-level middleware
- Built a searchable, sortable inventory table with low-stock threshold flagging and multi-level (Junior/Senior High) organization
- Added CSV report generation and a Chart.js dashboard summarizing inventory, borrowed, overdue, and damaged counts
- Built with Laravel 11, Inertia.js, Vue 3, MySQL

StaySwift — stayswift.pedialink.site or github.com/alexiseluzon/LaravelLuminaries-StaySwift-Hotel-Reservation-Application

- Built a booking engine with overlap detection, per-user duplicate-hold prevention, and auto-expiry of unpaid reservations via a scheduled task
- Integrated Stripe Checkout for payments, with server-side session verification and reservation status sync
- Implemented a multi-stage reservation lifecycle (pending, accepted, declined, ongoing, completed, cancelled, unattended) with admin reason-tracking for cancellations
- Built an admin dashboard with monthly revenue analytics (Chart.js) and full customer/room management
- Built with Laravel, CSS, JS, MySQL, hosted on Hostinger

EDUCATION

EXPERIENCE

Crew Trainer

Oct 2022 – Jun 2026

DCMF Golden Wings Development Corporation (McDonald's) • Philippines

- Trained and onboarded new crew members on standard operating procedures and food safety protocols
- Demonstrated leadership and communication skills by mentoring staff to meet performance standards

Kitchen Crew

Jun 2019 – Oct 2022

DCMF Golden Wings Development Corporation (McDonald's) • Philippines

- Maintained food safety and quality standards in a high-volume, fast-paced kitchen environment
- Collaborated with team members to meet daily operational targets efficiently